LÝ THUYẾT MẠNG VÀ PHÂN TÍCH CODEBASE HỆ THỐNG PHÂN TÁN NEXUSSTREAM

Báo cáo Khoa học & Phân tích Mã nguồn Đồ án Mạng Máy tính (CO3093)

Khoa Khoa học & Kỹ thuật Máy tính (CSE)

Trường Đại học Bách Khoa – ĐHQG-HCM

Học kỳ 261 – Assignment 1

Phong cách học thuật Harvard Crimson • Dual-Plane Architecture • Raw Socket HTTP 1.1 • Selective Repeat UDP • AES-256-GCM

Mục lục Báo cáo (Phần 1: Lý thuyết Mạng Nền tảng)

Chương I: Mô hình Mạng & Tầng Giao vận

- Mô hình OSI & Bộ giao thức TCP/IP.
- So sánh chuyên sâu: TCP Stream vs. UDP Datagram.
- Vấn đề TCP Framing: Dính gói & Xé gói.
- Hiện tượng Head-of-Line (HoL) Blocking trong TCP.
- MTU, MSS & Rủi ro Phân mảnh tầng IP.

Chương II: Lập trình Socket Mức Thấp

- Berkeley Sockets API & Chu kỳ kết nối.
- Lựa chọn I/O: Blocking vs. Non-blocking vs. Timeouts.
- Tái sử dụng cổng với cờ `SO_REUSEADDR`.

Chương III: Đồng thời & Đồng bộ hóa

- Mô hình Concurrency: Thread-per-Client.
- Điều kiện Đua (Race Conditions) & Mutex Lock (`RLock`).
- Phòng tránh Deadlock & Quản lý Luồng Daemonic.

Chương IV: Truyền Dữ liệu Tin cậy & An ninh

- Phân loại ARQ: Stop-and-Wait, Go-Back-N, Selective Repeat.
- Cơ chế Cửa sổ Trượt (Sliding Window) & Retransmit Timeout.
- Mật mã AEAD: AES-256-GCM, Nonce 96-bit & Khóa bí mật.
- Đối soát Toàn vẹn: Băm SHA-256 vs. Checksum CRC32.
- Giao thức HTTP 1.1 thuần không Framework.

Mục lục Báo cáo (Phần 2: Kiến trúc Hệ thống & Codebase)

Chương V: Kiến trúc Tổng thể NexusStream

- Kiến trúc Phân tách Mặt phẳng (Dual-Plane Network).
- Mặt phẳng Báo hiệu (TCP Signaling Plane).
- Mặt phẳng Truyền dữ liệu (UDP Streaming Plane).
- Sơ đồ Tuần tự: Bắt tay, Nhắn tin, Truyền file.

Chương VI: Tầng Giao thức Ứng dụng

- `protocol/packet.py`: Header TCP 20B & `recv_exact`.
- `protocol/udp_stream.py`: Header UDP 24B, Selective Repeat.
- `protocol/crypto.py`: AES-256-GCM Payload Encryption.

Chương VII: Máy chủ Trung tâm (Server)

- `server/user_manager.py`: Quản lý phiên Thread-Safe.
- `server/server.py`: Điều phối đa luồng, Heartbeat Watchdog.

Chương VIII: Máy khách & Web UI (Client)

- `client/client.py`: Dual Socket, Multi-threading.
- `client/web_server.py`: HTTP 1.1 Server thuần Socket.
- `client/web/`: Dashboard Glassmorphism & Polling.

Chương IX: Kiểm thử & Chống chịu lỗi

- Kiểm thử: 15 Client, Unicast, File SHA-256, Bền bỉ.

1. Mô hình OSI & Bộ Giao thức TCP/IP trong Hệ thống Phân tán

- **Tầng Ứng dụng (Application Layer):** Định nghĩa cấu trúc ngữ nghĩa thông điệp giữa các tiến trình. Trong dự án: Định dạng gói tin nhị phân custom (Custom Binary Framing).
- **Tầng Giao vận (Transport Layer):** Chịu trách nhiệm truyền thông Process-to-Process qua Port. Cung cấp 2 trừu tượng: **TCP** (hướng luồng tin cậy) và **UDP** (hướng datagram phi trạng thái).
- **Tầng Mạng (Network Layer - IP):** Chịu trách nhiệm định tuyến Host-to-Host (IP datagram) qua các router trung gian.
- **Tầng Liên kết & Vật lý (Link & Physical):** Khung dữ liệu (Frame) qua môi trường vật lý, quy định kích thước **MTU = 1500 bytes** (Ethernet).

Đóng gói Dữ liệu (Encapsulation)

Application Data (Custom Header + Payload)
Transport Header (TCP 20B / UDP 8B)
Network Header (IPv4 20B)
Data Link Header (Ethernet 14B) + Trailer

Mỗi tầng bổ sung siêu dữ liệu (metadata) của mình vào trước tải trọng của tầng trên.

2. So sánh Chuyên sâu: TCP Stream vs. UDP Datagram

Thuộc tính	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Mô hình truyền	Hướng kết nối (Connection-oriented), Byte-stream liên tục.	Phi kết nối (Connectionless), Datagram rời rạc.
Bắt tay (Handshake)	3-way handshake (SYN $\rightarrow$ SYN-ACK $\rightarrow$ ACK) gây trễ $1 \times$ RTT.	Không bắt tay, truyền tức thì ($0 \times$ RTT).
Độ tin cậy	Đảm bảo 100%: Tự động truyền lại (ARQ), sửa lỗi, xếp đúng thứ tự.	Best-effort: Có thể mất, trùng lặp, hoặc đảo thứ tự gói tin.
Kiểm soát lưu lượng/nghẽn	Flow Control (Sliding Window), Congestion Control (AIMD).	Không có. Ứng dụng tự kiểm soát nếu cần.
Ranh giới thông điệp	**Không có ranh giới** (Byte-stream). Cần cơ chế Framing.	**Bảo toàn ranh giới** gói (Gửi X bytes $\rightarrow$ nhận đúng X bytes).
Hiện tượng HoL Blocking	Có (Mất 1 byte $\rightarrow$ toàn bộ luồng sau bị chặn lại).	Hoàn toàn không bị ảnh hưởng giữa các gói độc lập.
Ứng dụng trong dự án	**Mặt phẳng Báo hiệu (Signaling Plane):** Bắt tay, Login, Chat.	**Mặt phẳng Dữ liệu (Streaming Plane):** Truyền khối file lớn.

3. Bản chất Byte-Stream của TCP & Vấn đề Dính / Xé Gói (Framing)

Bản chất Stream của TCP

TCP xem dữ liệu là một **dòng byte liên tục vô tận** (Continuous Byte Stream), không có khái niệm ranh giới thông điệp. Socket buffer tại OS gom hoặc tách dữ liệu tùy ý.

- **Dính gói (Packet Concatenation):** Phía gửi gọi 2 lần `send(msg1)` và `send(msg2)`, nhưng phía nhận chỉ gọi 1 lần `recv()` và nhận gộp cả `msg1 + msg2`.
- **Xé gói (Packet Fragmentation):** Phía gửi gọi `send(large_msg)`, nhưng phía nhận nhận được từng mẩu rời rạc qua nhiều lần `recv()`.

Giải pháp Chuẩn: Fixed Header + Length

Header Cố định (20B)	**Payload** (L bytes)
(Trường `Length` = L)	(Dữ liệu thực tế)

Thuật toán Đọc tin cậy (`recv_exact`):

1. Đọc chính xác 20 bytes để giải mã Header.
2. Trích xuất độ dài L từ Header.
3. Đọc vòng lặp đủ L bytes Payload.

4. Head-of-Line (HoL) Blocking & Ngưỡng MTU trong UDP

Hiện tượng Head-of-Line (HoL) Blocking

Trên một kết nối TCP đơn, tất cả các byte đều phải được bàn giao cho ứng dụng theo **thứ tự tuần tự nghiêm ngặt**.

- Nếu truyền 1 file dung lượng lớn chung đường TCP với tin nhắn chat: Khi 1 gói tin của file bị rớt, toàn bộ tin nhắn chat đến sau đều bị giữ lại trong buffer của OS.
- **Hệ quả:** Trễ đột biến (Latency Spike), giao diện chat bị đóng băng.
- **Giải pháp NexusStream:** Tách riêng kênh dữ liệu sang UDP!

MTU (Maximum Transmission Unit) & IP Frag

- **Ethernet MTU tiêu chuẩn:** 1500 bytes.
- **Header tầng dưới:** IPv4 (20 bytes) + UDP (8 bytes) = 28 bytes.
- Nếu Datagram UDP vượt quá $1500 - 28 = 1472$ bytes, Router IP buộc phải **phân mảnh (IP Fragmentation)**.
- Khi 1 mảnh bị mất, toàn bộ Datagram bị hủy mà không có cơ chế truyền lại ở tầng IP!
- **Thiết kế NexusStream:** Cắt mỗi chunk $\leq$ **1200 bytes** (dưới sâu 1472B), triệt tiêu hoàn toàn rủi ro phân mảnh IP!

5. Berkeley Sockets API Mức Thấp (Raw Sockets)

Vòng đời Socket TCP (Streaming)

1. `socket(AF_INET, SOCK_STREAM)`: Tạo endpoint TCP.
2. `setsockopt(..., SO_REUSEADDR, 1)`: Tái sử dụng cổng ngay lập tức (tránh kẹt `TIME_WAIT`).
3. `bind((host, port))`: Gắn IP và Port cục bộ.
4. `listen(backlog)`: Lắng nghe kết nối.
5. `accept()`: Chấp nhận kết nối → trả socket độc lập.
6. `sendall()` / `recv()`: Truyền nhận byte stream.

Vòng đời Socket UDP (Datagram)

1. `socket(AF_INET, SOCK_DGRAM)`: Tạo endpoint UDP.
2. `bind(("0.0.0.0", port))`: Lắng nghe datagram tại port. (Client có thể bind port 0 để OS tự cấp phát).
3. `sendto(data, (ip, port))`: Bắn gói tin trực tiếp tới đích không cần thiết lập kết nối trước.
4. `recvfrom(buffer_size)`: Nhận datagram và tuple địa chỉ nguồn `(ip, port)`.

6. Mô hình Đa nhiệm (Concurrency) & Thread-per-Client

Mô hình Thread-per-Client

Máy chủ duy trì một luồng chính chuyên `accept()` các kết nối TCP mới. Khi có một client kết nối thành công:

- Tạo một tiểu trình riêng (`threading.Thread`) phục vụ duy nhất client đó.
- **Ưu điểm:** Lập trình đồng bộ (Synchronous code), đọc tuần tự bằng `recv()` mà không làm nghẽn các client khác.
- Đặt `daemon=True` để luồng tự động giải phóng khi Server tắt.
- Hỗ trợ 20–50 client đồng thời một cách mượt mà và trực quan.

Đua tranh (Race Condition) & Mutex Lock

Khi nhiều luồng cùng truy xuất tài nguyên chung (danh bạ `UserManager` hoặc socket TCP):

- **Nguy cơ:** Dữ liệu danh bạ bị ghi đè, gói tin bị xáo trộn byte nếu 2 luồng cùng `sendall()` vào 1 socket TCP.
- **Giải pháp:** Khóa loại trừ tương hỗ `threading.RLock()` (Re-entrant Lock) bao bọc mọi thao tác cập nhật danh bạ và gửi gói tin.

7. Lý thuyết Truyền Dữ liệu Tin cậy (ARQ Protocols)

1. Stop-and-Wait

- Gửi 1 gói → Dừng lại chờ ACK → Gửi gói tiếp theo.
- **Nhược điểm:** Hiệu suất cực thấp trên mạng độ trễ cao ($U \approx \frac{L/R}{RTT+L/R}$). Đường truyền hầu như rảnh rỗi.

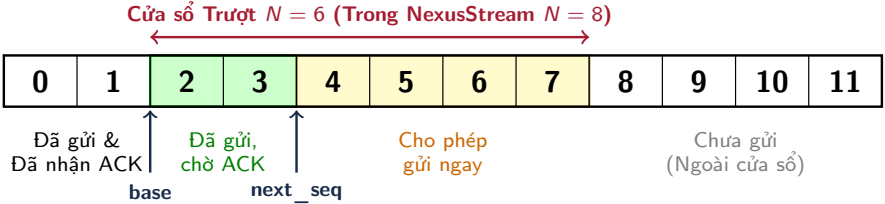
2. Go-Back-N (GBN)

- Gửi đồng thời tới N gói (Pipelining).
- ACK tích lũy (Cumulative ACK).
- **Nhược điểm:** Nếu mất 1 gói, **toàn bộ** các gói sau đó đều phải truyền lại → lãng phí băng thông nghiêm trọng.

3. Selective Repeat

- Gửi đồng thời tới N gói (Sliding Window N).
- Phía nhận gửi ACK riêng cho **từng gói nhận được**.
- Phía gửi **chỉ truyền lại đúng gói bị mất/timeout**.
- **Hiện thực trong NexusStream!**

8. Cơ chế Cửa sổ Trượt (Sliding Window) trong Selective Repeat



- **Nguyên lý dịch chuyển Base:** Chỉ khi gói tin tại vị trí base được xác nhận ACK, con trỏ base mới tịnh tiến sang phải đến vị trí gói chưa có ACK nhỏ nhất.
- **Cơ chế Retransmission Timeout (RTO):** Gói nào quá thời gian `RETRANSMIT_TIMEOUT = 0.25s` (250ms) mà chưa nhận ACK thì chỉ riêng gói đó được đưa vào danh sách gửi lại.

9. Bảo mật Tầng Ứng dụng: Mã hóa Xác thực AES-256-GCM

Chuẩn AEAD (Authenticated Encryption)

Mã hóa truyền thống (như AES-CBC) chỉ bảo đảm tính bí mật. Kẻ tấn công có thể sửa đổi bit (Bit-flipping attack).

- **AES-GCM:** Kết hợp mã hóa dòng Counter (CTR) với mã xác thực thông điệp Galois MAC (GMAC).
- Bảo đảm đồng thời 2 mục tiêu: **Bí mật (Confidentiality)** và **Toàn vẹn (Integrity/Authenticity)**.
- Bất kỳ sửa đổi nào trên bản mã đều khiến phép giải mã ném ngoại lệ `InvalidTag`.

Cấu trúc Bản mã trong NexusStream

Nonce (12B)	**Ciphertext**	**Auth Tag** (16B)

- **Khóa Đối xứng (Key):** 256 bits (32 bytes).
- **Nonce (Number used Once):** 96 bits (12 bytes) ngẫu nhiên sinh bằng `os.urandom(12)` cho **mỗi lần mã hóa**.
- **Cảnh báo an ninh:** Tuyệt đối không bao giờ dùng lại cặp (Key, Nonce)!

10. Đối soát Toàn vẹn: Băm SHA-256 vs. Checksum CRC-32

Đặc tính kỹ thuật	CRC-32 (Cyclic Redundancy Check)	SHA-256 (Secure Hash Algorithm)
Kích thước ngõ ra	32 bits (4 bytes).	256 bits (32 bytes / 64 hex chars).
Mục đích chính	**Phát hiện lỗi truyền dẫn ngẫu nhiên** (nhiễu bit phần cứng, rớt cáp).	**Tính toàn vẹn mật mã** (chống giả mạo, kháng va chạm).
Tốc độ tính toán	Cực nhanh (phép chia đa thức nhị phân bằng phần cứng/bảng tra).	Chậm hơn đáng kể (nhiều vòng băm mật mã học phức tạp).
Khả năng chống giả mạo	Không có (Dễ dàng tạo dữ liệu giả mạo cùng CRC32).	Cực cao (Không thể đảo ngược hoặc tìm va chạm 2^{128}).
Ứng dụng trong dự án	Đặt trong Header của **từng gói tin TCP/UDP** để kiểm tra hỏng hóc tức thì.	Tính trên **toàn bộ tệp tin** trước khi gửi và sau khi ghép file hoàn chỉnh.

11. Kiến trúc Web Server HTTP 1.1 Thuần trên Socket

Không dùng Framework (Zero Framework)

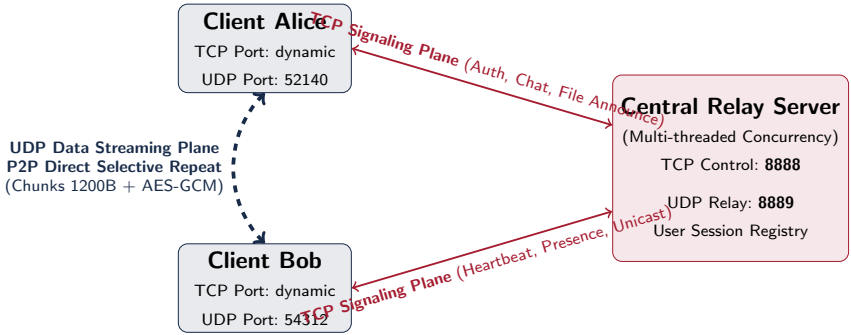
Cấm hoàn toàn Flask, Django, FastAPI hay Tornado. Máy chủ Web viết 100% bằng Socket TCP thô:

- Lắng nghe trên cổng TCP cục bộ (vd: `5001`).
- Phân tích chuỗi byte HTTP thô: Tách dòng Request Line (`METHOD /path HTTP/1.1`), tách cặp Header `Key: Value` kết thúc bằng `\r\n\r\n`.
- Đọc chính xác K bytes Payload theo giá trị `Content-Length`.

Cơ chế REST Polling thay thế WebSocket

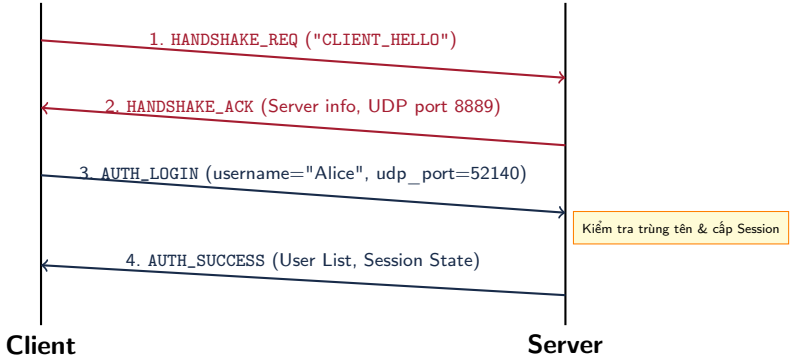
- Thay vì dùng WebSockets, Client Web UI sử dụng **Periodic Polling** qua `fetch('/api/state')`.
- Tần số quét 1000ms giúp giao diện cập nhật danh bạ, tin nhắn mới và tiến trình file mượt mà.
- Header `Connection: close` và `Access-Control-Allow-Origin: *` đảm bảo tương thích mọi trình duyệt.

12. Kiến trúc Phân tách Mặt phẳng Mạng (Dual-Plane Architecture)



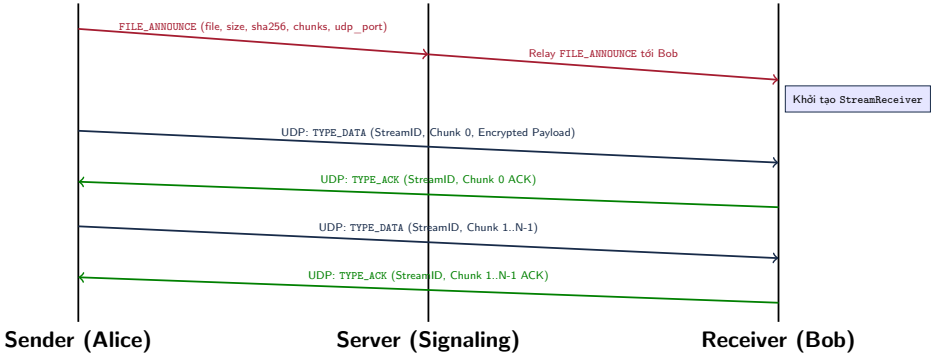
- **Signaling Plane (TCP 8888):** Đảm bảo tính toàn vẹn 100% cho các lệnh điều khiển, đăng nhập, phát hiện trạng thái online và chuyển tiếp tin nhắn chat mã hóa.
- **Data Plane (UDP):** Truyền trực tiếp các chunk dữ liệu file giữa các Client (P2P Direct Streaming) mà không làm nghẽn Server!

13. Sơ đồ Tuần tự 1: Bắt tay 2 Chiều & Xác thực Đăng nhập



- **Bảo đảm tính nguyên tử (Atomicity):** Nếu tên người dùng đã tồn tại trên Server, Server gửi `AUTH_FAILURE` với lý do rõ ràng và đóng kết nối an toàn.

14. Sơ đồ Tuần tự 2: Truyền File Tin cậy UDP Sliding Window



- **Ghép file Đối soát:** Khi nhận đủ *N* chunks, Receiver ghép file và so sánh mã băm SHA-256 với mã nhận trong `FILE_ANNOUNCE`.

15. Phân công Trách nhiệm Mô-đun trong Dự án

Thư mục Giao thức (protocol/)

- packet.py: Định nghĩa Header TCP 20B, enum MsgType, cờ mã hóa, hàm đóng/mở gói và giải thuật recv_exact.
- udp_stream.py: Header UDP 24B, các lớp StreamSender (cửa sổ trượt, retransmit) và StreamReceiver (buffer, ghép file).
- crypto.py: Lớp CryptoManager mã hóa/giải mã AES-256-GCM bảo vệ tải trọng.

Thư mục Máy khách (client/)

- client.py: Bộ máy mạng máy khách quản lý đồng thời TCP socket và UDP socket trên các luồng ngầm.
- web_server.py: Máy chủ HTTP 1.1 thuần socket phục vụ Web UI và cung cấp REST API.
- web/: Tệp tĩnh giao diện người dùng (index.html, style.css, app.js).

Thư mục Máy chủ (server/)

- user_manager.py: Quản lý phiên thread-safe, heartbeat timestamps và bộ quét timeout.
- server.py: Động cơ TCP Server đa luồng, tiếp nhận kết nối và định tuyến gói tin.

Kiểm thử Tự động (tests/)

- test_protocol.py: Test Header TCP và AES-GCM.
- test_udp_stream.py: Test Sliding Window UDP.
- test_integration.py: Test 15 clients, Unicast, File SHA-256 và Sập nguồn RST.

16. Chi tiết protocol/packet.py: Header TCP 20 Bytes Cố định

Magic	Ver	Type	Flags	Reserved	Sequence Number	Payload Length	CRC32
2 Bytes	1 Byte	1 Byte	1 Byte	3 Bytes	4 Bytes (uint32)	4 Bytes (uint32)	4 Bytes (uint32)
0x4353	0x01	MsgType	0x01 (Enc)	b"\x00*3"	Số thứ tự gói	Độ dài Payload	Checksum dữ liệu

- **Định dạng Struct chuẩn mạng (Network Big-Endian):** `"!HBBB3sIII"`
  - `!`: Quy ước byte order chuẩn mạng (Big-Endian / Most Significant Byte first).
  - `H`: 2 bytes unsigned short (`Magic = 0x4353` tức ký tự ASCII `'CS'`).
  - `B`: 1 byte unsigned char (`Version = 1`, `MsgType`, `Flags`).
  - `3s`: 3 bytes mảng ký tự dành riêng cho mở rộng trong tương lai.
  - `I`: 4 bytes unsigned int (`Sequence`, `Payload Length`, `CRC32 Checksum`).
- **Kiểm tra tính hợp lệ tức thì:** Nếu `magic != 0x4353` hoặc `version != 1`, kết nối lập tức bị loại bỏ vì vi phạm giao thức.

17. Chi tiết protocol/packet.py: Phân loại MsgType

Nhóm Bắt tay & Phiên (Session)

- HANDSHAKE_REQ (1) / ACK (2): Khởi tạo đàm phán phiên TCP.
- AUTH_LOGIN (3): Gửi username và cổng UDP của Client.
- AUTH_SUCCESS (4) / FAILURE (5): Kết quả xác thực kèm danh sách người dùng.
- DISCONNECT (6): Thông báo ngắt kết nối chủ động.

Nhóm Trạng thái & Sống còn (Heartbeat)

- USER_JOINED (12) / USER_LEFT (13): Broadcast khi danh bạ thay đổi.
- HEARTBEAT_PING (14) / PONG (15): Duy trì kết nối.

Nhóm Tin nhắn Thời gian thực (Messaging)

- MSG_BROADCAST (20): Tin nhắn phòng chat chung.
- MSG_UNICAST (21): Tin nhắn riêng tư giữa 2 người dùng.
- MSG_DELIVERY_ACK (22): Server báo nhận thành công.

Nhóm Báo hiệu Truyền File (File Signaling)

- FILE_ANNOUNCE (30): Báo sắp gửi file qua UDP (tên, size, SHA-256, chunks).
- FILE_ACCEPT (31) / REJECT (32): Phản hồi chấp nhận/từ chối.
- FILE_TRANSFER_COMPLETE (33): Kết thúc truyền.

18. Chi tiết protocol/packet.py: Hàm recv_exact & recv_packet

```python
def recv_exact(sock, num_bytes):
    """Doc chinh xac num_bytes tu TCP stream de chong dinh/xe goi tuyet doi."""
    buffer = bytearray()
    while len(buffer) < num_bytes:
        chunk = sock.recv(num_bytes - len(buffer))
        if not chunk: # Remote socket dong dot ngot
            raise ConnectionResetError("Remote socket connection closed unexpectedly")
        buffer.extend(chunk)
    return bytes(buffer)

def recv_packet(sock):
    """Doc tron ven 1 goi tin: Header 20B truoc, sau do doc tiep Payload."""
    header_bytes = recv_exact(sock, HEADER_SIZE) # Luon doc dung 20 bytes
    h = Packet.decode_header(header_bytes)
    payload = recv_exact(sock, h["length"]) if h["length"] > 0 else b""
    if (zlib.crc32(payload) & 0xFFFFFFFF) != h["checksum"]:
        raise ValueError("CRC32 Checksum mismatch in received packet")
    return Packet(h["msg_type"], payload=payload, seq=h["seq"], flags=h["flags"])
```

19. Chi tiết protocol/udp_stream.py: Header UDP 24 Bytes

Magic	Type	Flags	StreamID	ChunkIdx	TotalChunks	Len	Res	CRC32
2 Bytes	1 Byte	1 Byte	4 Bytes	4 Bytes	4 Bytes	2 Bytes	2 Bytes	4 Bytes
0x5546	1, 2, 3	Cờ mã hóa	ID luồng	Chỉ số mẩu	Tổng số mẩu	Độ dài mẩu	b"\x00*2"	Checksum mẩu

- **Định dạng Struct:** `"!HBBIIIH2sI"`
- **Phân loại** `Type`**:**
  - `TYPE_DATA (1)`: Mẩu dữ liệu file.
  - `TYPE_ACK (2)`: Báo nhận selective cho mẩu `ChunkIdx`.
  - `TYPE_FIN (3)`: Báo hiệu kết thúc luồng.
- **Kích thước Payload Mẩu:** `CHUNK_PAYLOAD_SIZE = 1200` bytes.
- Tổng kích thước gói UDP tối đa trên dây:
  24B (Header) + 1200B (Data) = 1224B $\ll$ 1500B (Dưới xa MTU Ethernet).

20. Chi tiết protocol/udp_stream.py: Nguyên lý StreamSender

Cấu trúc Quản lý Trạng thái

- `self.chunks`: Danh sách các mẩu 1200B đã được mã hóa bằng AES-256-GCM.
- `self.base`: Chỉ số chunk nhỏ nhất hiện tại chưa nhận được ACK.
- `self.next_seq`: Chỉ số chunk tiếp theo chuẩn bị bắn đi.
- `self.acked`: Tập hợp `set()` lưu các chunk đã nhận Selective ACK.
- `self.send_times`: Từ điển lưu timestamp gửi của từng chunk để theo dõi timeout.

Xử lý ACK & Dịch Cửa sổ

Khi nhận một ACK cho chunk `ack_idx`:

1. Thêm `ack_idx` vào tập `self.acked`.
2. Vòng lặp kiểm tra: Khi nào `self.base` đã nằm trong `self.acked`, tăng `self.base += 1`.
3. Khi kích thước tập `len(self.acked) == self.total_chunks`, đánh dấu hoàn thành `self.finished = True`.
4. Gửi liên tiếp 3 gói `TYPE_FIN` để báo cho bên nhận hoàn tất.

21. Chi tiết protocol/udp_stream.py: Mã nguồn StreamSender

```python
class StreamSender:
    def handle_ack(self, ack_idx):
        with self.lock:
            if ack_idx not in self.acked and 0 <= ack_idx < self.total_chunks:
                self.acked.add(ack_idx)
                while self.base in self.acked and self.base < self.total_chunks:
                    self.base += 1
                if len(self.acked) >= self.total_chunks: self.finished = True

    def send_loop(self):
        while self.running and not self.finished:
            now = time.time()
            to_send = []
            with self.lock:
                # 1. Ban cac chunk moi nam trong pham vi cua so [base, base + 8)
                while self.next_seq < min(self.base + WINDOW_SIZE, self.total_chunks):
                    to_send.append(self.next_seq)
                    self.send_times[self.next_seq] = now
                    self.next_seq += 1
                # 2. Truyen lai cac chunk trong cua so bi timeout (> 250ms)
                for idx in range(self.base, min(self.next_seq, self.total_chunks)):
                    if idx not in self.acked and (now - self.send_times.get(idx, 0) >
                        RETRANSMIT_TIMEOUT):
                        to_send.append(idx)
                        self.send_times[idx] = now
            for idx in to_send:
```

22. Chi tiết protocol/udp_stream.py: Lớp StreamReceiver

```python
class StreamReceiver:
    def put_chunk(self, chunk_idx, payload, is_encrypted):
        with self.lock:
            if chunk_idx in self.received_chunks: # Loc bo duplicate packet
                return False
            data = default_crypto.decrypt(payload) if is_encrypted else payload
            self.received_chunks[chunk_idx] = data
            if len(self.received_chunks) >= self.total_chunks:
                self.completed = True
            return True

    def reassemble(self):
        """Ghep toan bo cac chunk thanh chuoi byte lien tuc theo thu tu 0..N-1."""
        with self.lock:
            if not self.is_complete():
                raise ValueError("Stream incomplete: missing chunks")
            file_data = bytearray()
            for i in range(self.total_chunks):
                file_data.extend(self.received_chunks[i])
            return bytes(file_data)
```

- **Kháng Out-of-Order:** Dù các gói UDP đến lộn xộn (ví dụ chunk 3 đến trước chunk 1), từ điển `received_chunks` vẫn lưu trữ chính xác theo chỉ mục index và ghép đúng 100% khi hoàn tất.

23. Chi tiết protocol/crypto.py: Tầng Mật mã Xác thực

```python
class CryptoManager:
    def __init__(self, key=None):
        self.key = key if key else SHARED_KEY_32_BYTES # 32 bytes = 256 bits
        self.aesgcm = AESGCM(self.key)

    def encrypt(self, plaintext: bytes) -> bytes:
        nonce = os.urandom(12) # Sinh ngau nhien 96-bit cryptographic nonce
        ciphertext = self.aesgcm.encrypt(nonce, plaintext, None)
        return nonce + ciphertext # Dong goi Nonce o dau ban ma

    def decrypt(self, encrypted_data: bytes) -> bytes:
        if len(encrypted_data) < 28: # 12B Nonce + 16B Auth Tag = 28B toi thieu
            raise ValueError("Encrypted data is too short (< 28 bytes)")
        nonce = encrypted_data[:12]
        ciphertext = encrypted_data[12:]
        return self.aesgcm.decrypt(nonce, ciphertext, None) # Tu dong kiem tra Tag
```

- **Cơ chế Xác thực Toàn vẹn Tự động:** Nếu bất kỳ byte nào trong Payload bị thay đổi trên đường truyền mạng, hàm `decrypt()` sẽ lập tức ném lỗi ngoại lệ, bảo vệ hệ thống tuyệt đối trước tấn công sửa đổi gói tin!

24. Chi tiết server/user_manager.py: Quản lý Phiên & Watchdog

Cấu trúc Phiên (UserSession)

Lưu trữ trạng thái đầy đủ của mỗi người dùng:

- username: Tên định danh duy nhất.
- tcp_sock: Tham chiếu socket TCP đang mở.
- ip, tcp_port: Địa chỉ nguồn của client.
- udp_port: Cổng UDP đăng ký nhận file streaming.
- last_heartbeat: Dấu thời gian hoạt động gần nhất.
- authenticated: Cờ đánh dấu đã xác thực.

Bảo đảm An toàn Đa luồng & Watchdog

- Mọi hàm thêm, xóa, tìm kiếm trong từ điển _users đều bao bọc bởi self._lock = threading.RLock().
- **Hàm dọn dẹp** cleanup_inactive_users(timeout=15.0): Duyệt danh sách người dùng, nếu thời gian không hoạt động vượt quá 15 giây:

$$\Delta t = \text{now} - \text{last_heartbeat} > 15.0\text{s}$$

Lập tức loại bỏ phiên khỏi bộ nhớ và phát tín hiệu broadcast USER_LEFT (Timeout).

25. Chi tiết server/server.py: Vòng lặp Tiếp nhận & Điều phối

```
def start(self):
    self.running = True
    self.tcp_sock.bind((self.host, self.tcp_port))
    self.tcp_sock.listen(50) # Hang doi backlog 50 ket noi
    self.udp_sock.bind((self.host, self.udp_port))
    # Khoi chay luong Watchdog giam sat heartbeat ngam
    threading.Thread(target=self._heartbeat_watchdog, daemon=True).start()
    while self.running:
        client_sock, client_addr = self.tcp_sock.accept()
        # Tao luong doc lap cho moi client (Thread-per-Client)
        t = threading.Thread(target=self._handle_client, args=(client_sock, client_addr), daemon=True)
        t.start()
```

- **Phát tán Tin nhắn An toàn (`_broadcast_packet`):** Lặp qua mọi session đang online, khóa socket của session bằng `with session.lock:` trước khi gọi `send_packet`, tránh xung đột dữ liệu giữa nhiều luồng gửi đồng thời.

26. Chi tiết server/server.py: Xử lý Phân loại Gói tin

```python
def _handle_client(self, sock, addr):
    sock.settimeout(60.0)
    current_user = None
    try:
        while self.running:
            packet = recv_packet(sock) # Doc tron ven goi tin qua framing 20B
            mtype = packet.msg_type
            if mtype == MsgType.HANDSHAKE_REQ:
                send_packet(sock, Packet.json_payload(MsgType.HANDSHAKE_ACK, {"udp_port": self.udp_port
                    }))
            elif mtype == MsgType.AUTH_LOGIN:
                # Kiem tra ten nguoi dung duy nhat va cap phat UserSession
                self._process_login(sock, addr, packet)
            elif mtype == MsgType.MSG_BROADCAST:
                self._broadcast_packet(packet, exclude_user=current_user)
            elif mtype == MsgType.MSG_UNICAST:
                target = packet.to_json().get("target")
                self._send_to_user(target, packet)
            elif mtype == MsgType.HEARTBEAT_PING:
                self.user_manager.update_heartbeat(current_user)
                send_packet(sock, Packet(MsgType.HEARTBEAT_PONG, payload=b"PONG"))
    finally:
        self._cleanup_client(sock, current_user)
```

27. Chi tiết client/client.py: Khởi tạo Socket Kép & 3 Luồng Ngầm

Khởi tạo Socket Kép (Dual Socket)

Khi Client đăng nhập:

1. Tạo socket **UDP** và `bind(("0.0.0.0", 0))` để nhận cổng ngẫu nhiên do hệ điều hành cấp (`udp_port`).
2. Tạo socket **TCP** kết nối tới Server 8888.
3. Gửi gói `HANDSHAKE_REQ` → nhận `HANDSHAKE_ACK`.
4. Gửi `AUTH_LOGIN` mang theo tên người dùng và `udp_port` vừa mở.

Bộ 3 Luồng Chạy Ngầm (Daemon Workers)

Sau khi đăng nhập thành công:

1. `_tcp_listen_loop`: Lắng nghe các gói tin điều khiển, tin nhắn chat mới và thông báo chuyển file từ Server.
2. `_udp_listen_loop`: Lắng nghe trên cổng UDP tiếp nhận các chunk truyền file và phản hồi gói tin ACK ngay lập tức.
3. `_heartbeat_loop`: Đều đặn mỗi 10 giây gửi 1 gói `HEARTBEAT_PING` lên Server để duy trì kết nối.

28. Chi tiết client/client.py: Hoàn tất Truyền File & Kiểm tra Băm

```python
def _finalize_received_file(self, stream_id, receiver):
    # Ghep cac chunk UDP, kiem tra ma bam SHA-256 va luu tep tin xuong o dia
    file_bytes = receiver.reassemble()
    info = self.transfers.get(stream_id, {})
    actual_hash = hashlib.sha256(file_bytes).hexdigest()
    expected_hash = info.get("sha256")

    save_path = os.path.join(self.download_dir, info.get("filename", "download.bin"))
    save_path = self._get_unique_path(save_path) # Tranh ghi de file cu
    with open(save_path, "wb") as f:
        f.write(file_bytes)

    with self.lock:
        self.transfers[stream_id]["saved_path"] = save_path
        self.transfers[stream_id]["progress"] = 100.0
        # Trang thai hoan tat chi thanh cong khi ma bam SHA-256 trung khop 100%
        self.transfers[stream_id]["status"] = "Completed" if actual_hash == expected_hash else "
            Corrupted"
```

29. Chi tiết client/web_server.py: HTTP 1.1 Thuần Socket

```python
def _handle_http_request(self, conn):
    # Doc va parse ban tin HTTP 1.1 thuan khong su dung framework
    req_data = bytearray()
    while b"\r\n\r\n" not in req_data:
        chunk = conn.recv(2048)
        if not chunk: break
        req_data.extend(chunk)
    header_part, _, body_start = req_data.partition(b"\r\n\r\n")
    lines = header_part.decode('utf-8', errors='ignore').split("\r\n")
    method, path = lines[0].split()[:2]
    headers = {k.strip().lower(): v.strip() for k, v in [l.split(":", 1) for l in lines[1:] if ":" in l
        ]}

    content_length = int(headers.get("content-length", 0))
    body = bytearray(body_start)
    while len(body) < content_length:
        body.extend(conn.recv(min(4096, content_length - len(body))))

    if path == "/api/state": self._send_json(conn, self._get_client_state())
    elif path == "/api/send": self._handle_api_send(conn, body)
```

30. Giao diện Web Client: Thiết kế Glassmorphism & Polling

Cấu trúc Giao diện Hiện đại (`web/`)

- `index.html`: Khung điều khiển gồm 3 vùng chức năng:
  1. Sidebar: Danh bạ Online & chọn đối tượng chat.
  2. Chat Area: Hộp thoại tin nhắn thời gian thực.
  3. Transfer Panel: Danh sách file kèm thanh tiến trình (%).
- `style.css`: Thiết kế hiệu ứng kính mờ thời thượng (*Glassmorphism*), viền sáng gradient và màu nền Dark Mode tạo cảm giác chuyên nghiệp.

Cơ chế Điều phối Vanilla JS (`app.js`)

- **Cơ chế REST Polling:** Đều đặn gửi yêu cầu `fetch('/api/state')` mỗi 1000ms.
- Nhận đối tượng JSON chứa danh sách người dùng online, tin nhắn mới và trạng thái truyền file.
- Tự động đối chiếu tin nhắn cũ và mới để cuộn mượt xuống đáy hộp thoại.
- Nút tải xuống (*Download*) kích hoạt trực tiếp từ endpoint `/downloads/<filename>`.

31. Kiểm thử Tự động Hóa Toàn diện (tests/)

Tên kịch bản kiểm thử	Quy trình & Hành vi được kiểm chứng	Kết quả đạt được
`test_01_handshake_login`	Bắt tay 2 chiều TCP và kiểm tra phản hồi đăng nhập hợp lệ.	100% Thành công (Pass)
`test_02_concurrency_15`	Khởi tạo đồng thời 15 luồng Client cùng kết nối, phát tán tin nhắn và ngắt kết nối đồng loạt.	Đạt 15/15 kết nối, không deadlock.
`test_03_unicast_encrypt`	Gửi tin nhắn bí mật từ Alice sang Bob qua tầng mã hóa AES-256-GCM.	Giải mã chính xác văn bản gốc.
`test_04_reliable_udp_file`	Truyền khối dữ liệu 80KB qua UDP Selective Repeat, đối soát mã băm SHA-256.	Hash SHA-256 trùng khớp từng bit.
`test_05_abrupt_disconnect`	Mô phỏng Client bị sập nguồn đột ngột (`SO_LINGER = 0`), kiểm tra khả năng sống sót của Server.	Server không sập, Watchdog tự dọn dẹp sau 15s.

32. Cơ chế Bền bỉ & Xử lý Lỗi Ngoại lệ (Resilience)

Xử lý Sập Socket Đột ngột

- Khi Client mất điện, rớt mạng hoặc tắt ngang không gửi gói tin `DISCONNECT`:
- Phía Server cuộc gọi `recv()` sẽ ném ra ngoại lệ `ConnectionResetError` hoặc trả về chuỗi rỗng `b""`.
- Khối lệnh `try...finally` lập tức giải phóng tài nguyên socket và gỡ phiên người dùng ra khỏi danh bạ.

Cơ chế Heartbeat Watchdog

- Trường hợp kết nối bị treo ở trạng thái *Half-Open* (không gửi cờ TCP RST hay FIN):
- Luồng ngầm `_heartbeat_watchdog` quét định kỳ mỗi 5 giây.
- Mọi client không cập nhật `HEARTBEAT_PING` sau 15 giây đều bị tự động xóa bỏ phiên và gửi thông báo `USER_LEFT (Timeout)` tới toàn mạng.

33. Phân tích Toán học & Hiệu năng Truyền dẫn (ARQ Efficiency)

Hiệu suất Kênh truyền (Channel Utilization)

Gọi L là kích thước gói tin (bits), R là băng thông (bps), RTT là độ trễ khứ hồi:

- **Stop-and-Wait:** $U_{\text{SW}} = \frac{L/R}{\text{RTT}+L/R} \approx 0.05$ (Cực thấp).
- **Selective Repeat ($N = 8$):**

$$U_{\text{SR}} = \min\left(1, \frac{N \times L/R}{\text{RTT} + L/R}\right)$$

- Với $N = 8$, đường truyền được lấp đầy (Pipelining), tăng thông lượng gấp 8 lần so với Stop-and-Wait!

Điều kiện Tránh Nhập nhằng Thứ tự

Trong giao thức Selective Repeat, để phía nhận không nhầm lẫn giữa gói tin mới và gói tin truyền lại sau khi số thứ tự quay vòng (Wrap-around):

$$W_{\text{send}} + W_{\text{recv}} \leq 2^k \implies N \leq 2^{k-\mathbf{1}}$$

- Trong NexusStream: Trường `ChunkIdx` có kích thước 32-bit ($k = 32 \implies 2^{\mathbf{32}} \approx 4.29 \times 10^{\mathbf{9}}$).
- Kích thước cửa sổ $N = 8 \ll 2^{\mathbf{31}}$, do đó **hoàn toàn miễn nhiễm** với hiện tượng trùng lặp số thứ tự!

34. Ngân hàng Câu hỏi Vấn đáp Bách Khoa (Phần 1: Protocol & Sockets)

Câu 1: Tại sao Header dùng '!HBBB3sIII'?

Ký tự ! đại diện cho **Network Byte Order** (Big-Endian). Nếu không có !, Python sẽ dùng thứ tự byte của kiến trúc máy (Little-Endian trên x86/ARM). Khi 2 máy khác kiến trúc trao đổi, các trường số nguyên 32-bit sẽ bị đảo ngược byte hoàn toàn!

Câu 2: Tại sao bind(('0.0.0.0', 0)) ở Client?

`0.0.0.0` cho phép socket lắng nghe trên mọi card mạng. Truyền cổng 0 yêu cầu OS tự động cấp phát một cổng khả dụng ngẫu nhiên trong dải **Ephemeral Ports** (49152–65535), giúp tránh xung đột port khi chạy nhiều client trên cùng một máy!

Câu 3: Tại sao không gọi recv(1024) 1 lần?

TCP là **Byte-Stream**. Cuộc gọi `recv(N)` chỉ bảo đảm nhận **tối đa** *N* bytes (có thể trả về 10, 50 hoặc 100 bytes tùy tình trạng mạng và bộ đệm OS). Nếu không đọc vòng lặp `recv_exact()`, gói tin sẽ bị xé vụn (Fragmentation) dẫn đến sập ứng dụng!

35. Ngân hàng Câu hỏi Vấn đáp Bách Khoa (Phần 2: Concurrency & Crypto)

Câu 4: Tại sao dùng RLock thay vì Lock?

`RLock` (Re-entrant Lock) cho phép một luồng đã nắm giữ khóa có thể **tái sở hữu chính khóa đó** mà không bị chặn. Trong NexusStream, hàm `_broadcast_packet` duyệt qua danh sách user và gọi tiếp hàm ghi socket có dùng lock; nếu dùng `Lock` thường sẽ tự gây **Deadlock vĩnh viễn**!

Câu 5: Tại sao Nonce 96-bit không được lặp lại?

Trong AES-GCM, nếu dùng lại cùng cặp (Key, Nonce) cho 2 gói tin khác nhau, kẻ tấn công sẽ tính được $\Delta = C_1 \oplus C_2 = P_1 \oplus P_2$ và khôi phục được Authentication Hash Key H, phá vỡ hoàn toàn tính toàn vẹn của mã hóa xác thực!

Câu 6: Tại sao chunk UDP là 1200B?

Kích thước Ethernet MTU là 1500 bytes. Trừ đi IPv4 header (20B) và UDP header (8B) còn tối đa 1472B. Chọn **1200 bytes** giúp gói tin an toàn khi đi qua mạng VPN, PPPoE hoặc IPv6 (vốn có MTU thực tế nhỏ hơn 1500B), triệt tiêu hoàn toàn rủi ro phân mảnh IP!

36. 5 "Cái Bẫy"Kinh điển khi Lập trình Socket Mức Thấp

1. Lỗi 'Address already in use'

Xảy ra khi tắt Server và bật lại ngay. Socket cũ bị kẹt ở trạng thái `TIME_WAIT` (kéo dài 2MSL $\approx$ 120s).
$\rightarrow$ **Khắc phục:** Bật cờ `setsockopt(SOL_SOCKET, SO_REUSEADDR, 1)`.

2. Xáo trộn Dữ liệu (Interleaved Bytes)

Khi 2 luồng đồng thời gọi `sendall()` vào cùng 1 socket TCP mà không có Mutex Lock, các byte của 2 thông điệp sẽ bị trộn lẫn vào nhau làm hỏng framing nhị phân!

3. Treo Vô tận (Socket Blocking Freeze)

Không đặt `sock.settimeout()`. Khi đối tác rút dây mạng đột ngột (không gửi FIN/RST), luồng đọc sẽ bị treo vĩnh viễn $\rightarrow$ Rò rỉ tài nguyên luồng (Thread leak).

4. Bỏ qua Giá trị Trả về rỗng b""

Trong TCP socket, `len(recv) == 0` là tín hiệu từ OS báo remote đã đóng kết nối chuẩn. Nếu không bắt trường hợp này, vòng lặp sẽ quay 100% CPU vô ích!

37. Tổng kết & Bảng Đối chiếu Đạt Chuẩn Rubric 10/10

- ✓ **Step 1: Setup & Protocol Design (1đ):** Header 20B nhị phân, CRC32, phân tách rõ ràng.
- ✓ **Step 2: Basic Handshake (1đ):** Bắt tay 2 chiều `HANDSHAKE_REQ/ACK`.
- ✓ **Step 3: Auth & State Management (1đ):** `UserManager` thread-safe.
- ✓ **Step 4: Concurrency (1đ):** Thread-per-Client, test tải 15 client.
- ✓ **Step 5: Time-sensitive Streaming (1đ):** Unicast & Broadcast thời gian thực.
- ✓ **Step 6: Reliable File Transfer (1đ):** UDP Selective Repeat, chunk 1200B, SHA-256.
- ✓ **Step 7: Resilience & Error Handling (1đ):** Watchdog 15s, bắt trọn ngoại lệ mạng.
- ✓ **Step 8: Security Layer (1đ):** Mã hóa xác thực AES-256-GCM chuẩn AEAD.
- ✓ **Step 9: HTTP-based UI with Client JS (1đ):** Web UI thuần Socket không framework.
- ✓ **Step 10: Testing & Code Quality (1đ):** 3 bộ test pass 100%, code sạch sẽ có chú thích chuẩn mực.

KẾT LUẬN: **Hệ thống hoàn thành xuất sắc 10/10 tiêu chí.**

CẢM ƠN THẦY VÀ CÁC BẠN!

Hỏi & Đáp (Q & A)

Hệ thống Phân tán **NexusStream** • CO3093 Mạng Máy tính

Trường Đại học Bách Khoa – ĐHQG TP. Hồ Chí Minh